

成员操作手册（V1.3.0）

适用对象：使用 CodeBuddy + POM 流程的团队成员 **前置条件：**管理员已完成基础配置 **版本：**V1.3.0 | **日期：**2026-07-22 **你只需要关心：**什么场景用什么命令、Git怎么提交、怎么访问原型、怎么发布

一、角色说明

角色	谁来担任	做什么
管理员	Jojo	维护流程引擎、注册项目+授权成员、管理部署
技术成员	开发/产品技术	使用 CodeBuddy 命令走 POM 流程，开发项目
产品成员	产品/运营/设计	不用 CodeBuddy，只通过链接查看部署的原型和需求文档

二、首次入驻（一次性）

管理员先做

管理员执行一条命令完成注册：

```
/register-member --project=crm-portal --prefix=CRM --member=zhangsan
```

→ 自动完成：授权 + 端口分配 + 项目骨架 + 部署配置

你做

```
# 1. 克隆工作空间
git clone https://gitee.com/jojo180712/pom-workspace.git
cd pom-workspace

# 2. 配置 Git 身份
git config user.name "zhangsan"
git config user.email "zhangsan@example.com"

# 3. 打开 CodeBuddy，开始使用
# AI 自动识别你的身份（通过 git user.name 匹配）
```

三、日常开发流程

3.1 新项目从零开始

步骤1: 学习项目需求 (如有现有文档)

```
/learn-project --project=erp-portal --source=local-dir:./需求文档
```

→ AI 自动提取需求, 产出存入知识库

步骤2: 启动需求流

```
/init --project=erp-portal --type=新增 --desc=ERP门户v1.0
```

步骤3: 跟着 AI 走完流程 (AI 会引导你, 你只需在3个卡点确认)

需求分析 → 【确认①范围】 → 设计 → 架构 → 【确认②方案】
→ 开发 → 测试 → 验收 → 【确认③交付】

步骤4: 关闭版本 (自动部署+自动Git提交)

```
/close --project=erp-portal
```

→ 自动部署原型到 EdgeOne → 输出访问链接
→ 自动 git add + commit + push (你只需在 Gitee 点合并)

3.2 迭代已有项目

```
/init --project=erp-portal --type=迭代 --desc=新增退款功能
```

→ AI 自动加载上一版本数据, 你继续走流程
→ /close → 自动部署 v2.0.0 + 自动Git提交

3.3 变更需求

```
/change --project=erp-portal --desc=退款流程增加审核环节 --module=订单管理
```

→ AI 分析变更影响 → 更新文档 → 更新代码 → 回归测试 → 验收
→ /close → 自动部署 + 自动Git提交

3.4 修复 Bug

方式1: /fix (Bug专项, 完整闭环)

```
/fix --project=erp-portal --desc=客户列表分页问题 --module=客户管理
```

→ Bug分析 → 修复 → 回归验证 → 验收 → /close

方式2: /init --type=修复 (精简流程)

```
/init --project=erp-portal --type=修复 --desc=修复分页问题
```

→ 直接开发→测试→验收 → /close

命令选择速查

你的场景	用什么命令
新项目从零开始	<code>/init --type=新增</code>
已有项目加新功能	<code>/init --type=迭代</code>
改已有需求的流程/规则	<code>/change</code>
修Bug	<code>/fix</code> 或 <code>/init --type=修复</code>
学现有文档	<code>/learn-project</code>
学行业知识	<code>/learn-domain</code>

四、访问说明

4.1 本地开发访问

启动开发服务器后，用浏览器访问：

```
cd projects/erp-portal && npm run dev
→ http://localhost:{端口}
```

一个项目内有多多个业务端，通过路径区分：

以 AI-SCRM 项目（端口3333）为例：

http://localhost:3333/ops

→ 运营后台（PC端）

http://localhost:3333/tenant

→ 租户后台（PC端）

效果：打开不同路径 = 打开不同业务端

数据：所有端共享同一个浏览器数据库 → 运营后台创建的订单，租户后台立即能看到

持久：关闭浏览器再打开，数据还在；版本迭代数据不丢

4.2 多项目并行访问

不同项目用不同端口，互不干扰：

http://localhost:3333/...

→ AI-SCRM 项目

http://localhost:4000/...

→ erp-portal 项目（新项目从4000开始分配端口）

http://localhost:4001/...

→ crm-portal 项目

效果：不同项目的浏览器数据完全隔离，互不可见

4.3 部署后访问

`/close` 后原型自动部署到 EdgeOne Pages，生成访问链接：

原型地址：https://ai-scrm.edgeone.app/ai-scrm/v1.0.0/
需求文档：https://ai-scrm.edgeone.app/ai-scrm/v1.0.0/prd/

- 产品成员通过访问门户（管理员提供URL）查看所有项目版本
- 点击【访问原型】查看可交互的高保真原型
- 点击【查看PRD】查看HTML格式的需求文档

4.4 多端访问举例

以一个 SaaS 项目为例，运营后台发布版本，租户订阅下单，各端看到的数据：

本地开发（同一端口，路径区分端）：

http://localhost:3333/ops
→ 运营后台发布 v2.0 版本 → 数据写入浏览器数据库

http://localhost:3333/tenant

- 租户后台订阅 v2.0 → 下单 → 数据写入同一个浏览器数据库
- 运营后台刷新 → 看到租户的订单 + 订单状态（待付款→已付款→已开通）

效果：运营和租户在同一端口下，数据天然互通

持久：关闭浏览器再打开，订单数据还在

迭代：v1→v2→v3，数据一直延续不丢失

部署后（同一域名，路径区分项目+版本）：

https://ai-scrm.edgeone.app/ai-scrm/v1.0.0/ → v1.0 原型
https://ai-scrm.edgeone.app/ai-scrm/v1.1.0/ → v1.1 原型（新版本）

五、产出说明

5.1 发布高保真原型

`/close --project=erp-portal`

→ 自动执行：

- ① 版本冻结（Git Tag）
- ② 构建仿真原型（npm run build:sim）
- ③ 部署到 EdgeOne Pages
- ④ 输出访问 URL：https://ai-scrm.edgeone.app/erp-portal/v1.0.0/
- ⑤ 自动 Git 提交（你只需在 Gitee 点合并按钮）

→ 产品成员通过访问门户查看可交互的原型

手动重新部署（部署失败时）：

```
/deploy --project=erp-portal --retry
```

查看部署状态：

```
/deploy --status
```

5.2 生成 HTML 需求文档

需求分析阶段（/init 流程中）AI 自动生成：

- PRD 需求文档（Markdown格式）→ 存入 projects/{project}/docs/01-requirements/
- HTML 需求文档 → 存入 projects/{project}/docs/09-versions/{version}/index.html

/close 部署时自动复制到部署产物：

- <https://ai-scrm.edgeone.app/erp-portal/v1.0.0/prd/>

HTML需求文档包含：

- 版本历史 + 目录 + 背景目标 + 范围
- 功能需求列表（FN/UC）+ 业务规则（BR）+ 数据实体（ENT）
- 五类图（Mermaid渲染：泳道图/信息流/状态机/时序图/接口时序图）
- 集中配置（CONFIG）+ 统计指标（METRIC）+ 非功能需求（NFR）
- 验收标准

5.3 生成用例交互卡片

验收通过后，AI 自动执行交付标注（handoff）：

- 校验交互卡与 PRD 的 FN/UC 对应性
- 交互卡嵌入到原型页面中（可交互操作时弹出用例说明）
- 回传 BA 补充用例（如有变更）

/close 时自动归档：

- 用例卡片随版本快照归档到 docs/09-versions/{version}/
- 部署后的原型中可直接体验交互卡片

5.4 生成报告

```
/report --project=erp-portal --report-type=版本报告
```

- 生成版本报告（含需求覆盖/测试结果/验收结论/质量评分）

六、Git 协作

日常工作流

```
# 1. 每天开始前拉取最新代码
git checkout main
git pull origin main

# 2. 创建工作分支
git checkout -b zhangsan/erp-portal-v1

# 3. 开发（使用 CodeBuddy 命令）
# /init --project=erp-portal --type=新增 --desc=...
# ... 走完流程 ...

# 4. /close 后自动 git add + commit + push
# 你只需在 Gitee 上点合并按钮

# 5. 合并后清理分支
git checkout main
git pull origin main
git branch -d zhangsan/erp-portal-v1
```

V1.2.0起： `/close` 后自动执行 `git add + commit + push`，你只需在 Gitee 点合并按钮，无需手动 Git 操作。

多项目并行开发

```
# 为每个项目创建独立分支
git checkout -b zhangsan/erp-portal-v2    # erp-portal 迭代
git checkout main
git checkout -b zhangsan/crm-portal-v1    # crm-portal 新项目

# 分别提交（只提交对应项目的文件）
git checkout zhangsan/erp-portal-v2
git add projects/erp-portal/
git commit -m "feat(erp-portal): v2.0.0 迭代"
```

七、反馈提交

方式A：流程内反馈

```
/feedback --project=erp-portal --msg="需求分析阶段缺少业务规则模板"
```

方式B：Git Issue

在 Git 仓库创建 Issue, 标注 feedback / bug / enhancement 标签

方式C：直接告知管理员

管理员会记录到 problem-ledger → 评估影响 → 安排修复

/close 后，涉及流程改进的建议会自动收集，管理员审核后应用到流程引擎。

八、能做 / 不能做

能做

操作	命令
脑暴	/brainstorm --brainstorm-type=regular --project=xxx
启动需求	/init --project=xxx --type=新增\ 迭代\ 变更\ 修复 --desc=...
变更需求	/change --project=xxx --desc=... [--module=...] [--file=...]
修复Bug	/fix --project=xxx --desc=... [--module=...] [--bug-id=...]
学习项目	/learn-project --project=xxx --source=...
学习领域	/learn-domain --category=... --name=... --source=...
关闭版本（自动部署+自动Git）	/close --project=xxx
手动部署	/deploy --project=xxx
查看部署状态	/deploy --status
查看报告	/report --project=xxx --report-type=...
交付标注	/handoff --project=xxx
提交反馈	/feedback --project=xxx --msg=...

不能做

操作	原因
查看/修改 <code>.codebuddy/configs/</code>	流程引擎，管理员维护
查看/修改 <code>.codebuddy/knowledge/common/</code>	方法论知识库，管理员维护
修改 <code>deploy/</code>	部署配置，管理员维护
修改未授权的项目	只能操作管理员授权的项目

九、常见问题

Q1：成员有多个项目怎么管理？

每个项目有独立的目录、端口。通过 `--project` 参数切换。管理员通过授权控制你能访问哪些项目。

Q2：项目切换时数据会串吗？

不会。不同项目用不同的浏览器数据库名，数据完全隔离。

Q3：版本升级后数据还在吗？

在。同一项目不同版本共用同一个浏览器数据库，版本升级数据自动延续。关闭浏览器再打开数据也还在。

Q4：/init、/change、/fix 有什么区别？

命令	场景
<code>/init --type=新增</code>	新项目从零开始
<code>/init --type=迭代</code>	已有项目加新功能
<code>/change</code>	改已有需求的流程/规则
<code>/fix</code>	修Bug（完整闭环）
<code>/init --type=修复</code>	修Bug（精简流程）

Q5：部署失败怎么办？

部署失败不阻断版本冻结。执行 `/deploy --project=xxx --retry` 手动重试。

Q6：/close后还需要手动提交Git吗？

不需要。`/close` 后自动执行 `git add + commit + push`，你只需在 Gitee 上点合并按钮。

Q7：换电脑怎么办？

```
git clone https://gitee.com/jojo180712/pom-workspace.git
cd pom-workspace
git config user.name "zhangsan"
git config user.email "zhangsan@example.com"
cd projects/erp-portal && npm install
# 代码在 Git 中恢复，浏览器数据新电脑重新初始化
```

Q8：流程中AI问我影响范围/版本规划，我怎么操作？

AI 会呈现影响模块清单和建议方案，你只需选择： – 采纳建议 — 大多数情况选这个 – 进一步收窄 — 本期只做部分模块时选这个 – 补充遗漏 — 你知道有模块AI没扫到时选这个

如果影响面过大，AI 会建议拆分版本，你可以采纳/调整/不拆分。

十、速查卡

常用命令

场景	命令
启动开发服务器	cd projects/{project} && npm run dev
新项目	/init --project=xxx --type=新增 --desc=...
迭代版本	/init --project=xxx --type=迭代 --desc=...
变更需求	/change --project=xxx --desc=...
修复Bug	/fix --project=xxx --desc=...
关闭版本（自动部署+Git）	/close --project=xxx
手动部署	/deploy --project=xxx
查看部署状态	/deploy --status
查看报告	/report --project=xxx --report-type=...
提交反馈	/feedback --project=xxx --msg=...

流程卡点（只需3次确认）

卡点	阶段	操作
①	需求分析后	确认范围
②	架构评审后	确认方案
③	验收完成后	确认交付

访问地址

场景	地址
本地开发	http://localhost:{端口}/{端路径}
部署后原型	https://ai-scrm.edgeone.app/{project}/{version}/
部署后PRD	https://ai-scrm.edgeone.app/{project}/{version}/prd/

Git 速查

场景	命令
拉取最新	git pull origin main
创建分支	git checkout -b {member}/{project}-v{version}
/close自动提交	自动 git add + commit + push, 你只需在 Gitee 点合并
合并后清理	git checkout main && git pull && git branch -d {branch}